arXiv:2401.01755v1 [cs.SD] 3 Jan 2024

INCREMENTAL FASTPITCH: CHUNK-BASED HIGH QUALITY TEXT TO SPEECH

Muyang Du[1], *Chuan Liu*[1], *Junjie Lai*[1]

[1]NVIDIA Corporation

ABSTRACT

Parallel text-to-speech models have been widely applied for real-time speech synthesis, and they offer more controllability and a much faster synthesis process compared with conventional auto-regressive models. Although parallel models have benefits in many aspects, they become naturally unfit for incremental synthesis due to their fully parallel architecture such as transformer. In this work, we propose Incremental FastPitch, a novel FastPitch variant capable of incrementally producing high-quality Mel chunks by improving the architecture with chunk-based FFT blocks, training with receptive-field constrained chunk attention masks, and inference with fixed size past model states. Experimental results show that our proposal can produce speech quality comparable to the parallel FastPitch, with a significant lower latency that allows even lower response time for real-time speech applications.

***Index Terms*—** text-to-speech, speech synthesis, real-time, low-latency, streaming tts

1. INTRODUCTION

In recent years, Text-to-Speech (TTS) technology has witnessed remarkable advancements, enabling the generation of natural and expressive speech from text inputs. Neural TTS system primarily contains an acoustic model and a vocoder. It involves first converting the texts to Mel-spectrogram by acoustic models such as Tacotron 2[1], FastSpeech[2], FastPitch[3], GlowTTS[4], then converting the Mel feature to waveform by vocoders such as WaveNet[5], WaveRNN[6, 7], WaveGlow[8], and HiF-GAN[9]. Moreover, with the boost of real-time and streaming applications, there is an increasing demand for TTS systems capable of producing speech incrementally, also known as streaming TTS, to provide lower response latency for better user experience. For example, Samsung[10] proposed a low-latency streaming TTS system running on CPUs based on Tacotron 2 and LPCNet[11]. NVIDIA[12] also proposed a highly efficient streaming TTS pipeline running on GPUs based on BERT[13], Tacotron 2 and HiFi-GAN. Both of them uses auto-regressive acoustic model for incremental Mel generation.

Auto-regressive acoustic models such as Tacotron 2 is capable of producing natural speech by leveraging sequential generation to capture prosody and contextual dependencies. However, it suffers from slow inference due to the frame-by-frame generation process and susceptibility to over-generation and word-repeating artifacts due to unstable alignment learned between the input phonemes and output frames. In contrast, parallel acoustic models like such as FastPitch offers a faster inference process by producing complete Mel-spectrogram in one step. Additionally, it also shows benefits in providing the flexibility to manipulate pitch, duration, and speed of the synthetic speech as those metadata are pre-generated before decoding.

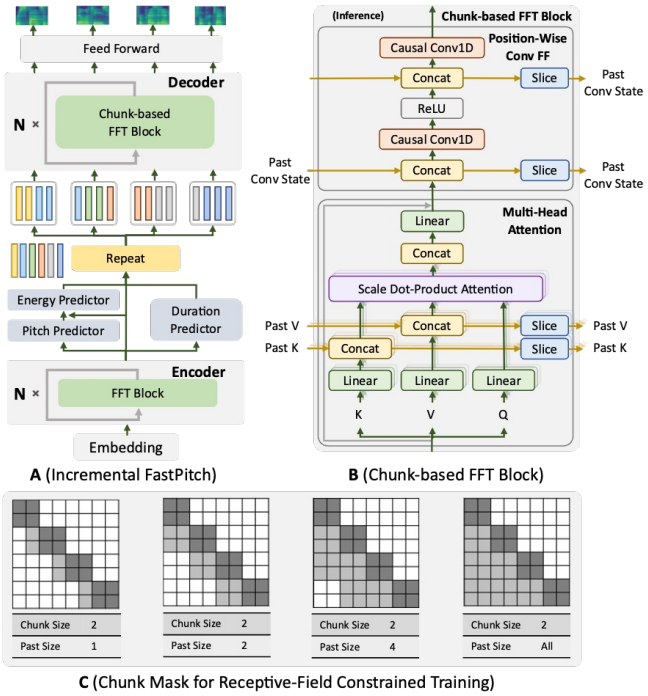

Fig. 1: Incremental FastPitch, Chunk-based FFT Block, and Chunk Mask for Receptive-Filed Constrained Training

Although parallel acoustic models offer many advantages, their model structure poses challenges for their use in incremental speech synthesis. For instance, FastPitch utilizes a transformer[14] decoder, wherein attention is computed across the entire encoded feature sequence to generate the Mel-spectrogram output. A straightforward method is to slice the encoded feature sequence into chunks and then decode each chunk into a corresponding Mel chunk. However, this approach compels the decoder to focus only on a chunk, re-

sulting in audible discontinuity at the edges of Mel chunks, even when overlapping between chunks is used. An alternative approach is to modify the model to use an auto-regressive decoder. However, this fails back to frame-by-frame generation, sacrificing the parallelism advantage. Therefore, an ideal decoder for incremental TTS should be able to incrementally generate Mel chunks while maintaining parallelism during the chunk generation process and keeping the computational complexity of each chunk consistent in the meantime.

Based on the above considerations, we present Incremental FastPitch, capable of producing high-quality Mel chunks while maintaining chunk generation parallelism and providing low response latency. We incorporate chunk-based FFT blocks with fixed-size attention state caching, which is crucial for transformer-based incremental TTS to avoid the computational complexity increases with synthetic length. We also utilize receptive-filed constrained training and investigate both the static and dynamic chunk masks, which is vital to align the model with limited receptive-filed inference.

2. METHOD

2.1. Incremental FastPitch

Figure 1A depicts the proposed Incremental FastPitch model, a variant of the parallel FastPitch. It takes a complete phoneme sequence as input and generates Mel-spectrogram incrementally, chunk-by-chunk, with each chunk contains a fixed number of Mel frames. Incremental FastPitch is equipped with the same encoder, energy predictor, pitch predictor, and duration predictor as the parallel FastPitch. However, the decoder of Incremental FastPitch is composed of a stack of chunk-based FFT blocks. In contrast to the decoder of parallel FastPitch that takes the entire upsampled unified feature $\bar{\boldsymbol{u}}$ as input and generate the entire Mel-spectrogram at once, The decoder of Incremental FastPitch first divide the $\bar{\boldsymbol{u}}$ to N chunks $[\bar{\boldsymbol{u}}_{\boldsymbol{1}}, \bar{\boldsymbol{u}}_{\boldsymbol{2}}, ..., \bar{\boldsymbol{u}}_{\boldsymbol{N}}]$, then convert one chunk $\bar{\boldsymbol{u}}_{\boldsymbol{i}}$ at a time to a chunk of Mel $\bar{\boldsymbol{y}}_{\boldsymbol{i}}$. During training, we apply a chunk-based attention mask on the decoder to help it adjust to the constrained receptive field in incremental inference, which we term it as the Receptive Field-Constrained Training.

2.2. Chunk-based FFT Block

Figure 1B illustrates the chunk-based FFT block, which contains a stack of a multi-head attention (MHA) block and a position-wise causal convolutional feed forward block. Compare with parallel FastPitch, the MHA block in the chunk-based FFT block requires two additional inputs: past key and past value, produced by itself during previous chunk generation. Instead of utilizing all the accumulated historical past keys and values from prior chunks, we employ fixed-size past key and value for inference by retaining only their tails. The past size maintains consistent throughout incremental generation, preventing an increase in computational complexity with the number of chunks. Although we impose an explicit past size limit, experiments shows that it is capable of encoding sufficient historical information for generating high-quality Mel. The calculation of MHA is defined as:

$$\begin{aligned} k_i^t &= \text{concat}(pk_i^{t-1}, KW_i^K) \\ v_i^t &= \text{concat}(pv_i^{t-1}, VW_i^V) \\ o_i^t &= \text{attention}(k_i^t, v^t, QW_i^Q) \\ o_M^t &= \text{concat}(o_1^t, ..., o_h^t)W^O \\ pk_i^t &= \text{tail_slice}(k_i^t, S_p) \\ pv_i^t &= \text{tail_slice}(v_i^t, S_p) \end{aligned} \tag{1}$$

where pk_i^{t-1} and pv_i^{t-1} are the past K and past V of head i from chunk $t-1$. k_i^t and v_i^t are the embedded K and V with the past concatenated along the time dimension for attention computation of head i at chunk t. o_M^t is the output of MHA block at chunk t. W_i^K, W_i^V, W_i^Q, and W^O are the trainable weights. S_p is the configurable fixed size of the past. pk_i^t and pv_i^t are obtained by slicing size S_p from the tail of k_i^t and v_i^t along the time dimension.

Similarly, the calculation of position-wise causal convolution feed forward block is defined as:

$$\begin{aligned} c_1^t &= \text{concat}(pc_1^{t-1}, o_M^t) \\ o_{c_1}^t &= \text{relu}(\text{causal_conv}(c_1^t)) \\ c_2^t &= \text{concat}(pc_2^{t-1}, o_{c_1}^t) \\ o_{c_2}^t &= \text{relu}(\text{causal_conv}(c_2^t)) \\ pc_1^t &= \text{tail_slice}(c_1^t, S_{c_1}) \\ pc_2^t &= \text{tail_slice}(c_2^t, S_{c_2}) \end{aligned} \tag{2}$$

where pc_1^{t-1} and pc_2^{t-1} are the past states of the two causal convolutional layers. Starting with pc_1^{t-1}, it's concatenated with o_M^t to yield c_1^t, serving as input for the first causal conv layer. Next, $o_{c_1}^t$, the output from the first causal conv layer, is concatenated with pc_2^{t-1} to generate c_2^t. This is then input to the second causal conv layer, resulting in the final output o_{c2}^t. Lastly, pc_1^t and pc_2^t are extracted by slicing sizes S_{c_1} and S_{c_2} from the tail of c_1^t and pc_2^t along the time dimension, respectively. Unlike the configurable S_p, we set S_{c_1} and S_{c_2} to their respective conv kernel sizes minus 1, which is adequate to attain equivalence with parallel inference.

2.3. Decoder Receptive Field Analysis

Figure 2 demonstrates the receptive filed of the proposed chunk-based decoder. For better visualization, we omit the positional-wise convolutional feed-forward blocks. The orange block at the top-right corner represents the final FFT output O_t of chunk t. The dark green MHA blocks are those whose multi-head attention, past key, and past value outputs contribute to O_t. The light green MHA blocks are those

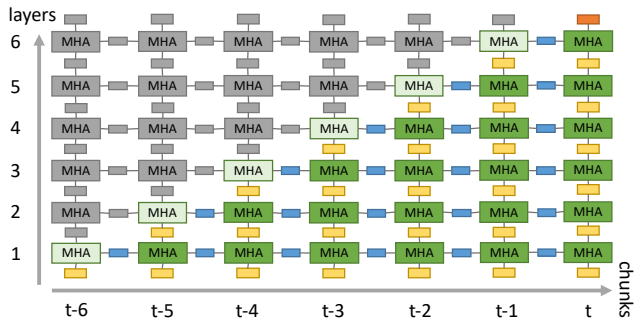

Fig. 2: Chunk-based decoder receptive field visualization.

whose past key and past value outputs contribute to O_t. Similarly, the blue blocks (past keys and past values) and the yellow blocks (inputs of green MHA blocks) are those who contribute to O_t. By feeding the fixed size past key and past value of chunk $t-1$ to each MHA block during chunk t generation, we can expand the receptive field of chunk t to several of its previous chunks without the need to explicitly feed those previous chunks as decoder input.

The receptive field $\mathcal{R}$ depends on the number of decoder layers and the size of past keys and past values, as given by:

$$\mathcal{R} = (N_d + \lfloor S_p/S_c \rfloor + 1) \cdot S_c \tag{3}$$

where N_d is the number of decoder layers, S_p is the size of past keys and past values, and S_c is the size of the chunk. The unit of $\mathcal{R}$ is the number of decoder frames. If S_p is less than or equal to S_c, then the past key and past value output by a MHA block only depends on the input of that MHA block, thus $\mathcal{R}$ simply equals to $(N_d+1)\cdot S_c$, same as shown in figure 2, whereas if S_p is greater than S_c, then the past key and past value of a MHA block at chunk t will also depends on the past keys and values of that MHA block at previous chunks, resulting in $\mathcal{R}$ grows linearly with the floor of S_p/S_c.

2.4. Receptive Field-Constrained Training

Given a limited decoder receptive field during inference, it becomes vital to align the decoder with this constraint during training. Therefore, we use the Receptive Field-Constrained Training by applying chunk-based attention mask to all the decoder layers. Figure 1C visualizes various attention masks with a given chunk size (dark grey) and different past sizes (light grey). An intuitive approach is to randomly select a chunk size and past size for dynamic mask creation for each text-audio training data pair within a batch. This approach is similar to the masks used in the WeNet[15, 16] ASR encoder. The dynamic mask can help the decoder generalize to diverse chunk and past sizes. However, most of the incremental system TTS employs a fixed chunk size for inference. Using a dynamic mask for training may potentially introduce a gap between training and inference. Therefore, we also investigate training with static masks that constructed using a fixed chunk size and past size during the training process.

3. EXPERIMENTS

3.1. Experimental Setup

Dataset. The Chinese Standard Mandarin Speech Corpus[17] released by DataBaker is used for both training and evaluation. It contains 10,000 48kHz 16bit audio clips of a single Mandarin female speaker and has a total of 12 hours with each audio clip contains a short sentence of 4.27 seconds on average. In our experiments, we downsample the corpus to 22.05kHz and 100 audio clips are reserved for evaluation.

Model & Acoustic Specifications. The proposed model parameters follow the open-source FastPitch implementation[18], except that we use causal convolution in the position-wise feed forward layers. The decoder is used to predict Mel-spectrogram with 80 frequency bins. It is generated through an FFT size of 1024, a hop length of 256 and a window length of 1024, applied to the normalized waveform. To enhance convergence speed and stability, the Mel values are standardized within a symmetrical range from -4 to 4.

Training & Evaluation. Our models are trained using the Adam optimizer[19] with batch size 8, initializing with a learning rate of 1e-4 and a weight decay of 1e-6. The experiments are performed on an NVIDIA RTX 6000 GPU, utilizing single precision and applying gradient clipping[20]. We use Mel-spectrogram distance (MSD) and mean opinion score (MOS) to measure the speech quality. To ensure the Mel-spectrograms of two audios are properly aligned for MSD calculation, we first use a trained parallel FastPitch to produce unified duration, pitch, and energy values for evaluation texts, then use these values to process the output feature of Incremental FastPitch encoder. Regarding the MOS, we synthesize waveform for evaluation with HiFi-GAN trained using the same dataset as FastPitch. Since we focus on optimizing acoustic model for incremental TTS, the vocoding process is non-incremental. For Incremental FastPitch, we concatenate all the Mel chunks to the complete Mel for vocoding. The MOS scores are collected through the assessment of 20 evaluation samples for each configuration by 10 Amazon MTurk listeners, who assign scores ranging from 1 to 5. For audio samples, please refer to GitHub page[1].

3.2. Discussion

3.2.1. Comparison of Static and Dynamic Chunk Masks

Figure 3 shows the Mel-spectrogram distance between the Incremental FastPitch and the parallel FastPitch. During inference, we use a fixed chunk size 30 for all the models. In the sub-figure A, the models are train with static chunk masks. The chunk sizes are fixed to 30 and past sizes are set to 0, 5, 15, 30, 60, 90, and all. We can observe that the smallest MSD of each model is often achieved when we use the same (or similar) chunk size and past size for training and inference.

[1] https://muyangdu.github.io/incremental-fastpitch

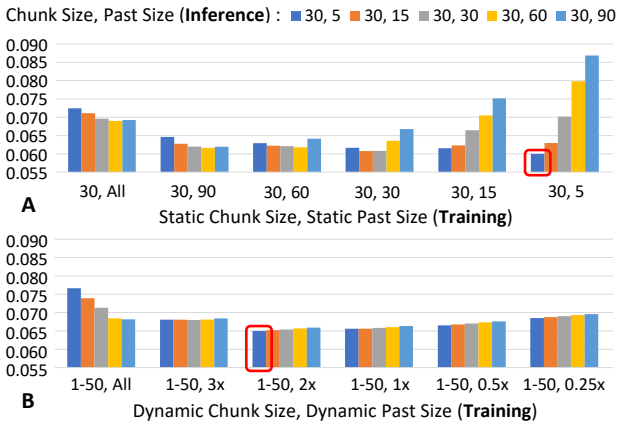

Fig. 3: MSD between the parallel FastPitch and the Incremental FastPitch trained with different types of masks, then inference with different chunk and past sizes. Each bar in the figure represents a specific (chunk size, past size) for **inference**. The horizontal axis describes the (chunk size, past size) used for **training**. **A**. Static Mask. **B**. Dynamic Mask.

The smallest MSD is achieved with past size 5 (red marked). Specifically, we find that if the model is trained with a small past size such as 5, it has a high MSD when inference with a big past size such as 90. On the contrary, if the model is trained with a big past size, it has a more stable MSD when inference with small past sizes. This observation suggests that even if the model is trained with a larger past context, it still learns to generate Mel chunk condition on nearby past contexts, rather than those far from the current chunk.

In the sub-figure B, the models are trained with dynamic chunk masks. The chunk sizes are randomly selected from range 1 to 50, and the past sizes are set to 0, 0.25, 0.5, 1, 2, 3 times of the selected chunk size and all. We observe that the MSD are more stable and similar if the inference past size changes, compared with static mask. The smallest MSD is achieved when we use 2 times of the randomly selected chunk size as the past size. However, the MSD of the dynamic chunk mask models are generally higher than the static chunk mask models. This observation confirms our suspicion raised in subsection 2.4 that dynamic mask training can introduce a training inference mismatch. Based on the above analysis, it is suggested to use a static mask for the best quality if the inference chunk and past sizes can be known in advance.

3.2.2. *Visualized Ablation Study*

We perform visualized ablation study to investigate the necessity of using past key value and past conv state. Figure 4 shows the synthetic Mel-spectrograms of parallel FastPitch and Incremental FastPitch. We can observe that the Incremental FastPitch can generate Mel with almost no observable difference compared with parallel FastPitch. However, if either the past key value or the conv state is removed, apparent discontinuation can be found between adjacent Mel chunks.

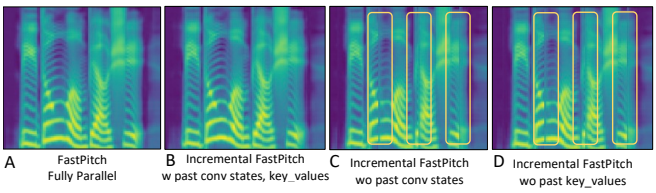

Fig. 4: Mel-spectrogram Visualization.

3.2.3. *Evaluation of Speech Quality and Performance*

To study the audible speech quality of both the static (S) and dynamic (D) mask trained Incremental FastPitch, we perform listening tests on the best S and D models selected based on the MSD analysis (marked as red in figure 3). As shown in table 1, we find that Incremental FastPitch is capable of producing high quality speech comparable with the parallel FastPitch. Furthermore, the score of D model is only slightly lower than the S model, although the D model has a 8.3% higher MSD compared with the S model. This result shows that the audible difference of the S and D model is barely noticeable, especially with the compensation of vocoder.

Table 1: Mean opinion score (MOS) with 95% CI, real time factor (RTF), and latency (ms) comparison on evaluation set.

Model	MOS	Latency	RTF
Par. FastPitch	4.185 ± 0.043	125.77	0.029
Inc. FastPitch (S)	4.178 ± 0.047	30.35	0.045
Inc. FastPitch (D)	4.145 ± 0.052		
Ground Truth	4.545 ± 0.039	-	-

Table 1 also displays RTF and latency. For Incremental FastPitch, RTF is defined as dividing the last chunk's latency by the audio duration, and latency corresponds to the first chunk's latency. The S and D model shares the same inference process. We find that Incremental FastPitch has a higher RTF but is still able to achieve around 22× real-time as it maintains the parallelism of chunk generation. Notably, it has a significantly lower latency compared to parallel FastPitch.

4. CONCLUSIONS

In this work, we propose Incremental FastPitch, capable of incrementally generating high-quality Mel chunks with low latency while maintaining chunk generation parallelism and consistent computation complexity. We improve the decoder with chunk-based FFT blocks that use fixed size state caching to maintain Mel continuity across chunks. We further experiment with multiple masking configurations of receptive-filed constrained training for adapting model to limited receptive filed inference. Experiments show that our proposal can produce speech quality comparable to the parallel baseline, with a significant lower latency that allows even lower response time for real-time speech synthesis.

5. REFERENCES

[1] Jonathan Shen, Ruoming Pang, Ron J Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, Rj Skerrv-Ryan, et al., "Natural tts synthesis by conditioning wavenet on mel spectrogram predictions," in *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2018, pp. 4779–4783.

[2] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu, "Fastspeech: Fast, robust and controllable text to speech," *Advances in neural information processing systems*, vol. 32, 2019.

[3] Adrian Łańcucki, "Fastpitch: Parallel text-to-speech with pitch prediction," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6588–6592.

[4] Jaehyeon Kim, Sungwon Kim, Jungil Kong, and Sungroh Yoon, "Glow-tts: A generative flow for text-to-speech via monotonic alignment search," *Advances in Neural Information Processing Systems*, vol. 33, pp. 8067–8077, 2020.

[5] A van den Oord, S Dieleman, H Zen, K Simonyan, O Vinyals, A Graves, N Kalchbrenner, AW Senior, and K Kavukcuoglu, "Wavenet: A generative model for raw audio, corr, vol. abs/1609.03499," 2017.

[6] Nal Kalchbrenner, Erich Elsen, Karen Simonyan, Seb Noury, Norman Casagrande, Edward Lockhart, Florian Stimberg, Aaron Oord, Sander Dieleman, and Koray Kavukcuoglu, "Efficient neural audio synthesis," in *International Conference on Machine Learning*. PMLR, 2018, pp. 2410–2419.

[7] Muyang Du, Chuan Liu, Jiaxing Qi, and Junjie Lai, "Improving WaveRNN with Heuristic Dynamic Blending for Fast and High-Quality GPU Vocoding," in *Proc. INTERSPEECH 2023*, 2023, pp. 4344–4348.

[8] Ryan Prenger, Rafael Valle, and Bryan Catanzaro, "Waveglow: A flow-based generative network for speech synthesis," in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 3617–3621.

[9] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae, "Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis," *Advances in Neural Information Processing Systems*, vol. 33, pp. 17022–17033, 2020.

[10] Nikolaos Ellinas, Georgios Vamvoukakis, Konstantinos Markopoulos, Aimilios Chalamandaris, Georgia Maniati, Panos Kakoulidis, Spyros Raptis, June Sig Sung, Hyoungmin Park, and Pirros Tsiakoulis, "High quality streaming speech synthesis with low, sentence-length-independent latency," pp. 2022–2026, ISCA.

[11] Jean-Marc Valin and Jan Skoglund, "Lpcnet: Improving neural speech synthesis through linear prediction," in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 5891–5895.

[12] Muyang Du, Chuan Liu, Jiaxing Qi, and Junjie Lai, "Efficient incremental text-to-speech on gpus," in *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. IEEE, 2023, pp. 1422–1428.

[13] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of naacL-HLT*, 2019, vol. 1, p. 2.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[15] Zhuoyuan Yao, Di Wu, Xiong Wang, Binbin Zhang, Fan Yu, Chao Yang, Zhendong Peng, Xiaoyu Chen, Lei Xie, and Xin Lei, "WeNet: Production Oriented Streaming and Non-Streaming End-to-End Speech Recognition Toolkit," in *Proc. Interspeech 2021*, 2021, pp. 4054–4058.

[16] Binbin Zhang, Di Wu, Zhendong Peng, Xingchen Song, Zhuoyuan Yao, Hang Lv, Lei Xie, Chao Yang, Fuping Pan, and Jianwei Niu, "WeNet 2.0: More Productive End-to-End Speech Recognition Toolkit," in *Proc. Interspeech 2022*, 2022, pp. 1661–1665.

[17] Databaker, "Chinese standard mandarin speech corpus," `https://www.data-baker.com/open_source.html`, 2023, Accessed: September 3, 2023.

[18] NVIDIA, "Fastpitch," `https://github.com/NVIDIA/DeepLearningExamples/tree/master/PyTorch/SpeechSynthesis/FastPitch`, 2023, Accessed: September 3, 2023.

[19] Diederik P. Kingma and Jimmy Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun, Eds., 2015.

[20] Xiangyi Chen, Steven Z Wu, and Mingyi Hong, "Understanding gradient clipping in private sgd: A geometric perspective," *Advances in Neural Information Processing Systems*, vol. 33, pp. 13773–13782, 2020.